M1 Assessment – Case Study Based Questions with C# Solutions

1: Employee Salary Management

Description:

Case Study: Employee Salary Management System

Specifications:

Class: Salary
Data Member:
Dictionary<string, int> empList

Methods:
totalSalary() : int
getSalary(string designation) : string
updateSalary(string designation, int newSalary) : void

Task:

1. Calculate total salary.
2. Fetch salary by designation.
3. Update salary for a designation.

2: Speed Validation Using Custom Exception

Description:

Case Study: Vehicle Speed Validation

Specifications:

Class: CarSpeed
Data Member:
speed : string

Class: SpeedInvalidException

Class: CarSpeedImplementation
Method:
setCarSpeed(CarSpeed sp, int spd) : string

Task:

1. Validate speed between 30 and 90.
2. Throw custom exception if invalid.

3: Offer on Brands (LINQ)

Description:

Case Study: Car Brand Offer System

Specifications:

Class: Model
Fields:
ModelName : string
CarSpeed : int

Class: Implementation
Methods:
GetModelName(List<Model>) : List<string>
GetModelInfo(List<Model>, string, int) : Model

Task:

1. Fetch all model names.
2. Fetch model details using LINQ.

4: Harry's Assignment – String Operations

Description:

Case Study: String Manipulation System

Specifications:

Class: StringPlay
Data Members:
convert : int
max : int

Class: StringMethods
Methods:
ConvertToInt(StringPlay, string) : int
GetMax(StringPlay, string, char) : int

Task:

1. Convert string to integer.
2. Count character occurrences.

5: Country Code Validation

Description:

Case Study: Country Code Repository System

Specifications:

Class Definitions:
Class: Repository
Method:
static GetCountryName(string countryCode)
Return Type: string
Throws: InvalidCodeException

Class: RepositoryImplementation
Method:
static GetCountry(string countryCode)
Return Type: string
Visibility: public
Throws: InvalidCodeException

Class: InvalidCodeException

Task:

1. Return "India" if country code is between 70 and 99.
2. Return "US" if country code is 908.
3. Return "Dial somewhere outside of USA" if country code is 011.
4. Throw InvalidCodeException for all other cases.

6: Project Almanac

Description:

Case Study: Agent Project Management System

Specifications:

Class: Almanac
Data Member:
Dictionary<string, List<string>> projects

Methods:
assignProject(string name, string project) : string
currentProjects(string name) : List<string>
finishProject(string name, string project) : string

Task:

1. Assign projects to agents.
2. Fetch current projects of an agent.
3. Finish/remove a project.

7: All Spanish

Description:

Case Study: Dish Name Operations

Specifications:

Class: Dish
Data Member:
dishName : string

Class: DishTest
Methods:
addYummyToName(List<Dish>, string) : List<Dish>
count(List<Dish>, string) : long

Task:

1. Append given string to dish names.
2. Count dishes containing given string.

8: Hard Work

Description:

Case Study: Employee Bonus System

Specifications:

Class: Employee
Data Members:
name : string
projectName : string
workingHrs : int
bonus : int

Methods:
setBonus() : string
checkName() : string

Task:

1. Calculate bonus based on project and hours.
2. Return output based on employee name.

9. FilmFare

Description:

Case Study: FilmFare Award Eligibility System

Specifications:

Class: Rating
Data Members:
imdbRating : int
nominee : int

Class: Validator
Methods:
CanBeConsideredForTheAward(Rating) : string
SendInvite(Rating) : string

Custom Exception:
MovieRatingException

Task:

1. Throw exception if imdbRating < 7.
2. Throw exception if nominee < 4.
3. Otherwise consider for award.

10. Student Scholarship Scheme

Description:

Case Study: Student Scholarship Scheme

Specifications:

Class: Student
Fields:
studentName, studentId, studentScore, scholarshipScheme

Class: ScholarshipImpl
Methods:
AddStudent(Student)
GetStudentDetails(string)
DeleteStudent(int)

Task:

1. Assign scholarship based on score.
2. Filter students by scheme.

3. Delete student by id.

11. Make it Pangram

Description:
Case Study: Pangram Validation

Specifications:
Class: Pangram
Method:
MinimumAlphabet(string) : int

Task:
1. Count distinct alphabets.
2. Return missing alphabets count.

12. Candidate Validation System

Description:
Case Study: Candidate Eligibility System

Specifications:
Class: Candidate
Fields:
name, totalRating, totalContest

Class: Validator
Methods:
Eligible(Candidate) : string
SendInvite(Candidate) : string

Custom Exception:
CriteriaMismatchException

Task:
1. Validate rating and contest.
2. Send invite if eligible.

13. CouponDunia

Description:

Case Study: Product Coupon Validation System

You are developing a coupon validation system for an e-commerce platform.

Specifications:

Class: Product
Data Members:
name : string
price : double
coupon : string

Constructor:
Product(string name, double price, string coupon)

Class: Validator
Method Definitions:
ValidateCoupon(Product p) : string
NetPrice(Product p) : double

Custom Exception:
InvalidCouponException

Task:
1. Validate coupon format as <productName>-<discount>.
2. Product name must match (case-insensitive).
3. Discount must be between 10 and 25.
4. Calculate net price after applying discount.

14. Indistractable

Description:

Case Study: Book Cost Analysis System

Specifications:

Class: Book
Data Members:
bookName : string
bookCost : int

Class: BookImplementation
Methods:
GetNameOfBooks(List<Book>) : string

SumCostOfAllBooks(List<Book>) : int
GetMax(List<Book>) : int

Task:
1. Display book name and cost.
2. Calculate total cost.
3. Find maximum book cost.

15. Numeric Operations

Description:
Case Study: Numeric List Operations

Specifications:
Class: Source
Methods:
Sum(List<int>) : int
GetItemAtIndex(List<int>, int) : int?
SplitAndReverse(List<int>) : List<int>

Task:
1. Calculate sum of elements.
2. Return element at index or null.
3. Split list and reverse halves.

16. Vehicle To Purchase

Description:
Case Study: Vehicle Price Analysis

Specifications:
Class: Vehicle
Data Members:
name : string
price : double

Class: VehicleImplementation
Methods:
SumOfPrices(List<Vehicle>) : double
GetVehicleList(List<Vehicle>) : List<string>
MaxPrice(List<Vehicle>) : double

Task:
1. Sum vehicle prices.

2. Get vehicles priced above 25000.
3. Find maximum price.
(All using Lambda expressions)

17. Students in Classroom

Description:

Case Study: Student List Operations

You are working on a classroom management system. You are given a list of student names and need to perform various list-based operations using C#.

Specifications:

Class Definitions:
Class: MainClass
Method Definitions:
1. changeOccurrence(List<string> a, string m, string n)
   Return Type: List<string>
   Visibility: public
2. listIndex(List<string> list)
   Return Type: string
   Visibility: public
3. listAfter(List<string> a, string m, string n)
   Return Type: List<string>
   Visibility: public

Task:

1. Replace all occurrences of string m with string n.
2. Return the element present at index 0 of the list.
3. Insert string n immediately after every occurrence of string m.

18. What is an Age

Description:

Case Study: Person Age Analysis

You are given a list of persons with name and age details. Using Stream/LINQ operations, perform various operations without using loops.

Specifications:

Class: Person
Data Members:
name : string
age : int
Constructor:
Person(string name, int age)

Class: StreamImplementation
Method Definitions:

1. sumAge(List<Person> list)
   Return Type: int
2. printName(List<Person> list)
   Return Type: List<string>
3. printAge(List<Person> list)
   Return Type: List<int>

Task:
1. Create Person class with private variables.
2. Use LINQ to calculate sum of ages above 50.
3. Return list of all person names.
4. Return list of all person ages.
5. Do not use loops or conditional statements.

19. Person (Submit Details)

Description:

Case Study: Person Detail Validation System
A system validates personal details like date of birth and email using custom exceptions.

Specifications:

Class: Person
Data Members:
name : string
dateOfBirth : string
email : string

Class: Implementation
Method Definitions:
1. validator(Person details)
   Return Type: string
2. submitDetails(Person details)
   Return Type: string

Custom Exceptions:
InvalidDateException
InvalidEmailException

Task:

1. Validate date format (dd-mm-yyyy) and year < 2000.
2. Validate email domain as @doselect.com.

3. Throw appropriate custom exceptions.
4. Return proper status messages based on validation.

20. Salary Calculator

Description:

Case Study: Employee Salary Validation

You are required to validate employee salary details and calculate total monthly salary.

Specifications:

Class: SalaryData
Data Members:
name : string
daysInMonth : int
salary : double

Class: Validator
Method Definitions:
1. validateSalaryData(SalaryData s)
   Return Type: string
2. getTotalSalary(SalaryData s)
   Return Type: double

Custom Exceptions:
InvalidDayException
InvalidSalaryException

Task:

1. Validate days as 28, 30, or 31.
2. Validate salary range between 0 and 10,00,000.
3. Throw custom exceptions if validation fails.
4. Calculate total salary if data is valid.